

A Catalogue of Evaluation Metrics for LLM-Based Multi-Agent Frameworks in Software Engineering

Ingrid Lima

State University of Ceará, Brazil
Fortaleza, Brazil
ing.vieira@aluno.uece.br

Anderson Martins Gomes

State University of Ceará, Brazil
Fortaleza, Brazil
anderson.martins@aluno.uece.br

Vitor Linhares

State University of Ceará, Brazil
Fortaleza, Brazil
vitor.fontenele@aluno.uece.br

Paulo Henrique Maia

State University of Ceara, Brazil
Fortaleza, Brazil
pauloh.maia@uece.br

Abstract

LLM-based Multi-Agent (LMA) frameworks have recently gained traction in software engineering, attracting attention due to their potential to enhance productivity by automating tasks such as code generation, testing, and quality assurance. However, evaluation practices in this area remain fragmented due to the lack of standardised methodologies. Frameworks often rely on self-defined or inconsistent metrics, hindering reproducibility and making frameworks appear optimal within their own settings, which obscures the true state of the art and can produce artificially inflated performance results. To address these challenges, this study conducts a comprehensive analysis of evaluation metrics used by state-of-the-art LMA frameworks, revealing inconsistencies and conceptual gaps. We propose 12 novel metrics, combine them with 26 existing ones, resulting in a structured catalogue of 38 metrics across four technical categories: Outcome, Process, Product, and Framework. These contributions provide a structured foundation for rigorous, reproducible LMA framework evaluation, enabling direct and meaningful comparisons between frameworks and supporting the systematic advancement of LMA frameworks for software engineering.

CCS Concepts

• **Software and its engineering** → **Software design engineering; Software performance; Software verification and validation; Computing methodologies** → **Multi-agent systems.**

Keywords

LLM-based multi-agent frameworks, multi-agent frameworks, software engineering, evaluation metrics, LMA frameworks assessment, reproducible evaluation, metrics catalogue

ACM Reference Format:

Ingrid Lima, Vitor Linhares, Anderson Martins Gomes, and Paulo Henrique Maia. 2026. A Catalogue of Evaluation Metrics for LLM-Based Multi-Agent Frameworks in Software Engineering. In *International Workshop on Agent Engineering (AGENT '26)*, April 12–18, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3786167.3788430>



This work is licensed under a Creative Commons Attribution 4.0 International License. *AGENT '26, Rio de Janeiro, Brazil*

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2399-5/26/04
<https://doi.org/10.1145/3786167.3788430>

1 INTRODUCTION

LLM-Based Multi-Agent (LMA) frameworks (or systems) have gained significant attention for their potential in solving complex real-world tasks through collaboration among intelligent agents [4]. Within this context, software engineering is particularly suitable for LMA frameworks, where typically specialized agents (e.g., programmer, reviewer, tester) interact via natural language to achieve shared goals through distributed reasoning and task decomposition, as in [7]. These systems span multiple software development phases, from requirements engineering to code generation, testing, and maintenance. Various multi-agent frameworks have emerged for this purpose, such as ChatDev [16], MetaGPT [5], and Agile-Coder [12], each implementing different coordination strategies and evaluation methods. However, as the number of frameworks grows, so does the fragmentation in how their performance is evaluated. Current frameworks often rely on heterogeneous or self-defined metrics, employ inconsistent evaluation pipelines, and sometimes conflate or rename equivalent measures—making each system appear optimal within its own scope and hindering meaningful comparison. Moreover, some studies inappropriately compare frameworks from different categories, such as software-oriented systems (e.g., ChatDev) versus reasoning-oriented ones (e.g., MAS-GPT [20]), despite fundamental differences in their objectives and evaluation contexts. This practice, combined with the frequent use of identical evaluation concepts under different names, complicates the establishment of a shared and coherent metric structure.

To address these issues, we present a comprehensive catalogue of evaluation metrics for LMA frameworks in software engineering. Derived from a systematic analysis of state-of-the-art systems, the catalogue organizes 38 metrics—including 12 novel ones—into four categories: *Outcome* (software performance quality), *Process* (development workflow efficiency), *Product* (code integrity), and *Framework* (usability and human involvement). This categorisation aims to provide researchers with a structured reference point for identifying, selecting, and comparing evaluation metrics and frameworks. Our main contributions are threefold: (i) the catalogue of metrics, organised into four conceptual dimensions; (ii) a set of new metrics to improve the comparability, reproducibility, and methodological rigour of assessing LMA systems; and (iii) the identification and discussion of recurrent issues in current evaluation practices, such as the comparison of non-equivalent systems (e.g., software-oriented vs. reasoning-oriented frameworks).

The remainder of this paper is organised as follows: Section 2 presents the methodology, detailing the four-phase workflow adopted to review existing systems and metrics. Section 3 discusses the results and catalogue, analysing which metrics were included or excluded, and reflecting on naming inconsistencies and overlapping measures. Section 4 outlines the conclusion and future work, summarising the study’s contributions and proposing directions for continued research on standardised evaluation of LMA frameworks.

2 METHODOLOGY

To address these challenges, we developed a four-phase methodological workflow, illustrated in Figure 1, designed to systematically review, identify, and organise evaluation metrics used in LLM-based multi-agent systems for software engineering.

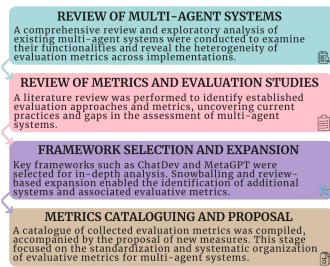


Figure 1: Overview of the methodological workflow.

Phase 1 — Review of Multi-Agent Systems

A comprehensive review and exploratory analysis were conducted on existing LLM-based multi-agent frameworks designed for software engineering. This stage focused on identifying system functionalities, domains of application, and evaluation practices. Prominent frameworks [16] [5] were examined to characterise the diversity of goals and architectures to generate functional software artifacts. Frameworks that did not employ multi-agent collaboration mechanisms were excluded. The reviewed frameworks represent the state-of-the-art in multi-agent LLM-based software engineering and are further detailed in the catalogue and discussion sections.

Phase 2 — Review of Metrics and Evaluation Studies

In the second phase, a targeted literature review was performed to identify prior works on the evaluation of LLM-based and multi-agent systems. This included empirical studies, technical reports, and framework documentation. Metrics were examined both in their formal definitions and implicit occurrences (e.g., within execution logs, appendices, or supplementary material). During this stage, we also mapped the main benchmarks recurrently adopted in the literature—such as *HumanEval* [12], *MBPP* [12], and their variants. Although datasets such as *MATH* [20] are widely used, they are not equally representative of all system categories, as they favour reasoning-orientated agents rather than those optimised for software generation. Therefore, our analysis focused on software engineering tasks and benchmarks that are most aligned with code-centric activities.

This phase revealed significant methodological inconsistencies and gaps, especially the absence of standardised evaluation pipelines and the conflation of conceptual definitions. The field of multimodal agents is experiencing rapid growth, with research efforts expanding at an unprecedented pace [13]. However, this oversight presents a critical gap in understanding their effectiveness in real-world deployments, particularly when communicating with humans. These observations reinforce the need for structured and transparent methodologies to evaluate LMA systems consistently across contexts.

Phase 3 — Framework Selection and Expansion

Building upon the prior phases, a focused selection of representative systems was carried out. Starting from [4], we employed a snowballing technique to identify additional frameworks based on citation relevance, experimental reproducibility, and methodological clarity. Only frameworks explicitly capable of generating or executing code for software engineering tasks were retained, while those outside this scope, such as *MobileSteward* [10], were excluded. Priority was given to well-cited systems considered benchmarks in the literature, ensuring representativeness and methodological consistency.

Phase 4 — Metrics Cataloguing and Proposal

The collected metrics were consolidated into a comprehensive catalogue, systematically classified into four conceptual categories: Outcome, Process, Product and Framework. Beyond organising existing measures, we proposed additional metrics to address aspects frequently discussed in the literature but often informalized or absent in current evaluations. These include dimensions such as collaboration efficiency, adaptability to evolving tasks, human-in-the-loop influence, and overall system usability. These proposed metrics aim to reduce fragmentation and provide a foundation for more rigorous, reproducible, and comparable assessments. The resulting catalogue serves both as a reference resource and as a methodological foundation for future studies, enabling consistent evaluation, fair comparison, and informed development of LLM-based multi-agent systems for software engineering.

This approach aligns with critical analyses, which highlights the lack of reproducible evaluation and the overestimation of system performance in isolated or self-defined settings. By identifying and formalising both existing and previously implicit qualitative dimensions, this study consolidates current knowledge while opening new avenues for understanding evaluation paradigms and methodological trade-offs in LLM-based multi-agent systems for software engineering, and helping guide the development of future multi-agent frameworks.

3 DISCUSSION

This section presents the catalogue of evaluation metrics (Table 1) and discusses its structure and relevance.

The inclusion and exclusion of frameworks followed three strict criteria, and only those frameworks satisfying all three were retained. (1) Code generation and executability: frameworks that do not generate or execute software code were excluded. Although multi-agent systems can serve a variety of purposes, the software

Table 1: Catalogue of Metrics for Software Development Agents and Reference Systems.

Category	Metric Name	Description	Reference
Outcome	Completeness	Measures the software's ability to fulfill code completion in software development, quantified as the percentage of software without any "placeholder" code snippets.	ChatDev, Croto, IER
Outcome	Executability ¹	Assesses the software's ability to run correctly within a compilation environment, quantified as the percentage of software that compiles successfully and can run directly.	ChatDev, Croto, IER
Outcome	Executability ²	Evaluates from a human perspective whether the software application is a complete failure or meets the task specifications to some extent, using a score from "failure" to "flawless".	MetaGPT, AgileGen
Outcome	Executability ^{3†}	Assesses the generated software against expected requirements to determine the success rate in meeting them, given by the ratio between requirements met and total requirements.	AgileCoder
Outcome	Success Rate [†]	Denotes the number of tasks successfully completed by the agent divided by the total number of tasks.	AutoGen, TOP
Outcome	Consistency	Measures how closely the generated software aligns with the original requirement description, quantified as the cosine distance between embeddings of text and code.	ChatDev, Croto, IER
Outcome	Quality	Assesses the overall software quality, computed as the product of completeness, executability, and consistency.	ChatDev, Croto, IER
Outcome	Pass@k	Evaluates functional accuracy of the top- <i>k</i> generated codes.	MetaGPT, AgileCoder, AgileGen
Outcome	Pass@1	Evaluates functional accuracy considering <i>k</i> = 1.	MetaGPT, AgileCoder, AgileGen, SOA
Outcome	Human Revision Cost	Refers to times of manual code corrections, with problems like package import errors, incorrect class names, or incomplete reference paths.	MetaGPT
Outcome	K-executions to success	Measures the number of complete generation–execution cycles required for a functional program that meets minimal task requirements.	NEW
Outcome	Ease of Use	Evaluates the generated software's ease of use via a Likert scale.	NEW
Outcome	Error-free Execution	Assesses the program's ability to run without errors.	MegaAgent
Outcome	CodeBLEU	Automatic code evaluation metric capturing syntax, semantics, data flow, and structure.	AgileGen
Outcome	Sounds Good	A generic boolean metric indicating whether a generated artifact meets a minimal set of structural and technological requirements specified in the prompt, based on a static analysis of the files.	NEW
Process	Time & Duration (s)	Total task execution time, in seconds.	All
Process	Cost & Total Tokens	Cost of execution, given by the number of tokens consumed (input + output).	All
Process	Number of Utterances	Number of messages exchanged between agents.	ChatDev, Croto
Process	Number of Self-Reflection	Number of times agents reflected upon the output.	ChatDev, Croto
Process	Productivity	Token usage divided by lines of code, representing token consumption per line.	MetaGPT
Process	Input Token	Number of input tokens consumed during execution.	All
Process	Output Token	Number of output tokens consumed during execution.	All
Process	Time / Agent (s)	Total execution time divided by number of agents used.	MegaAgent
Process	Rework Ratio	Proportion of code modified, rewritten, or discarded during self-reflection or critique phases.	NEW
Process	Number of Human Clarifications	Total number of clarification requests made by agents to human participants.	NEW
Process	Human Clarification Score	Quality of human clarifications, measured by Likert scale.	NEW
Process	Iteration Count	Total iterative cycles until stopping condition; each iteration = one production–verification loop.	NEW
Process	Time per Iteration	Time divided by Iteration Count.	NEW
Process	Tokens per Agent	Tracks the number of input/output tokens processed per agent.	NEW
Process	Pro_Code	Ratio of code tokens to total tokens, defined as Code/(Input + Output Tokens).	AgileGen
Product	Number of chars and tokens of Pure Code	Number of characters and tokens of code, excluding comments and docstrings.	SOA
Product	Code Amount per Function	Code generated by each agent function.	SOA
Product	Messages / LOC	Ratio between exchanged messages and lines of code in final output.	NEW
Product	Amount of Dead Code	Number of unused modules, classes, or functions.	NEW
Product	Number of Errors	Total number of errors in generated programs that fail to run.	AgileCoder
Framework	UEQ	Set of end-user questions to evaluate generated applications.	AgileGen
Framework	Human Involvement	Whether and how the framework allows human participation during execution.	AutoGen, MetaGPT, AgileGen
Framework	Framework Ease of Use	Evaluates framework usability using a Likert scale.	NEW

Executability (versions 1, 2, and 3): Different versions of the Executability metric measure executability from alternative perspectives.

†: Metrics with different names (Executability³ and Success Rate) that measure the same concept.

development life cycle primarily focuses on systems capable of producing and running code. (2) Single-agent limitation: systems employing a single autonomous agent, such as AutoGPT [19], were excluded, as their architecture lacks the collaborative and distributed characteristics of multi-agent systems, which have been shown to provide advantages in complex software engineering scenarios [4]. (3) Lack of applicable metrics: frameworks that do not explicitly define or propose evaluation metrics relevant to software development tasks were also discarded; conceptual or model-centric systems without measurable evaluation constructs fall outside the scope of this catalogue. Table 2 provides the analysed frameworks, including both selected and discarded ones, along with the criteria, if any, explaining why a framework was excluded.

As previously discussed, the catalogue organises evaluation metrics into four categories, each capturing a complementary aspect of LLM-based multi-agent frameworks. **Outcome:** Metrics assess

the tangible results by the framework (e.g., quality and completeness). **Process:** Metrics evaluate the efficiency and dynamics of the software generation workflow (e.g., token usage, execution time). **Product:** Metrics analyse the properties and integrity of produced artifacts (e.g., number of errors, amount of dead code). **Framework:** Metrics focus on the system itself (e.g., support for human involvement, ease of use).

For clarity and conciseness, several framework names appearing in the catalogue and in Table 2 have been abbreviated. Specifically, we use SOA to refer to *Self-Organized Agents* [6], IER for *Iterative Experience Refinement of Software-Developing Agents* [15], ToP for *Think-on-Process* [9], MLC to denote *Multi-LLM Cooperation* [8], and Croto for *Cross-Team Orchestration* [1]. These abbreviations are used consistently throughout this paper to improve readability without loss of specificity.

During catalogue construction, it became evident that identifying evaluation metrics is not a uniform process across frameworks.

Table 2: Inclusion of frameworks and reason for exclusion.

Framework	Included	Reason
AgileCoder	✓	-
AgileGen [21]	✓	-
AgentMesh [7]	✗	3
AtomAgents [2]	✗	1
Autogen [18]	✓	-
AutoGPT	✗	2
BabyAGI [11]	✗	2
ChatDev	✓	-
Croto	✓	-
Generative Agents [14]	✗	1, 3
IER	✓	-
MAS-GPT	✗	3
MegaAgent [17]	✓	-
MetaGPT	✓	-
MobileSteward [10]	✗	1, 3
MLC	✗	1, 3
PlotGen [3]	✗	3
SOA	✓	-
ToP	✓	-

In some studies (e.g., ChatDev, MetaGPT), metrics are clearly presented and systematically measured, was in others, they are implicit, embedded within logs, supplementary materials, or even inferred through result tables. In particular, the SOA framework demonstrates this challenge: although it proposes valuable dimensions, its metrics are not explicitly formalised, requiring careful interpretation of the paper to reconstruct the underlying measures.

Another relevant aspect observed during the analysis concerns the fragmentation of evaluation practices across frameworks. Although our catalogue focuses only on metrics from the selected frameworks, this phenomenon was also observed among the discarded ones, indicating a broader lack of methodological consistency in the field. Due to the absence of standardised evaluation protocols, we identified metrics sharing the same name but assessing distinct aspects, as well as metrics with different names referring to nearly identical constructs.

A particularly notable case involves frameworks that measure the same phenomenon in divergent ways—such as through percentages, discrete counts, or boolean indicators. For example, Error-Free Execution in MegaAgent and Executability¹ in ChatDev, Croto, and IER represent conceptually equivalent aspects of code operability but are quantified differently: as a boolean in MegaAgent and as a percentage in the others. While a unified naming convention could improve comparability, defining normative standards for metric nomenclature or computation lies beyond the scope of this work. With the exception of the new metrics explicitly proposed

by us, this catalog intentionally preserves the original definitions and formulations adopted by each framework, aiming to faithfully reflect current evaluation practices rather than impose prescriptive conventions.

In addition to cataloguing existing metrics, this study introduces twelve new measures to capture evaluation dimensions that are widely relevant yet often implicit in current LMA frameworks. **Outcome** metrics, such as *K-Executions to Success*, user-perceived *Ease of Use* and *Sounds Good* aim to capture whether the results generated by the framework meet minimal structural and usability expectations and how reliably functional outcomes are achieved. **Process** metrics, namely *Rework Ratio*, *Iteration Count*, *Time per Iteration*, *Tokens per Agent*, *Number of Human Clarifications* and *Human Clarification Score* focus on development dynamics beyond final success, with the first four characterising resource consumption and convergence speed and the last two formalizing how often and how effectively human-in-the-loop interaction occurs (for frameworks that allow it). **Product** metrics, such as *Messages/LOC* and *Amount of Dead Code*, evaluate the structural quality of the generated software artifacts by quantifying unused code and the amount of inter-agent communication required to produce the final output. By assessing the code itself, these metrics can show problems that are hidden by Outcome metrics, such as programs that run (good Outcome), but contain unused function or redundant modules. Finally, the **Framework** dimension is extended by the metric *Framework Ease of Use*, which assesses how easily users can interact with and adopt a given LMA framework. More detailed explanations with examples of use of each of the metrics have been made available¹. Furthermore, to ensure long-term relevance in this rapidly expanding field, the catalogue is maintained as an open-source resource in a GitHub repository. This allows for continuous versioning and community contributions, ensuring the artifact remains 'living' and updated as new frameworks and metrics emerge.

It is worth noting that traditional software engineering metrics were not identified among the assessment practices of the analysed frameworks. However, we consider these conventional measures—such as cyclomatic and cognitive complexity, code duplication, code smells, and test coverage—to be highly complementary and relevant for a comprehensive evaluation of the generated systems. These metrics were not included in this catalogue because they are not novel contributions and were not explicitly assessed by the reviewed frameworks. Nevertheless, we highlight their importance and encourage their systematic inclusion, particularly when evaluating the **Product** dimension of LLM-based multi-agent frameworks. Overall, these observations underscore the need for systematic evaluation practices and validate the structure, relevance and content of the presented catalogue.

4 CONCLUSION AND FUTURE WORK

Current LLM-based multi-agent frameworks for software engineering remain fragmented in their evaluation methodologies, leading to imprecise and often biased assessments. Frameworks are frequently compared using heterogeneous datasets or selectively defined metrics that highlight their own strengths while overlooking

¹Extended metric definitions and illustrative examples, are available [here](#).

unmeasured yet critical aspects. To mitigate this issue, we take an initial step toward standardisation by consolidating and cataloguing the metrics currently employed in state-of-the-art frameworks, while also proposing new ones to capture overlooked dimensions of performance. This catalogue offers a structured foundation for future evaluation and comparative studies.

Looking ahead, we plan to systematically validate each identified metric to assess its relevance and applicability across contexts. Building on this catalogue, we aim to perform controlled cross-framework comparisons and explore less-studied frameworks, refining our proposed metrics toward a standardised, reproducible, and conceptually grounded evaluation methodology for LLM-based multi-agent systems in software development.

Acknowledgments

This work received partial funding from CNPq-Brazil, Universal grant 404406/2023-8.

References

- [1] Zhuoyun Du, Chen Qian, Wei Liu, Zihao Xie, Yifei Wang, Rennai Qiu, Yufan Dang, Weize Chen, Cheng Yang, Ye Tian, et al. 2025. Multi-agent collaboration via cross-team orchestration. In *Findings of the Association for Computational Linguistics: ACL 2025*. 10386–10406.
- [2] Alireza Ghafarollahi and Markus J. Buehler. 2024. AtomAgents: Alloy design and discovery through physics-aware multi-modal multi-agent artificial intelligence. *arXiv:2407.10022 [cs.AI]* <https://arxiv.org/abs/2407.10022>
- [3] Kanika Goswami, Puneet Mathur, Ryan Rossi, and Franck Dernoncourt. 2025. Plotgen: Multi-agent llm-based scientific data visualization via multimodal feedback. *arXiv preprint arXiv:2502.00988* (2025).
- [4] Junda He, Christoph Treude, and David Lo. 2025. LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision, and the Road Ahead. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025), 1–30.
- [5] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiwu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. 2023. MetaGPT: Meta programming for a multi-agent collaborative framework. In *The Twelfth International Conference on Learning Representations*.
- [6] Yoichi Ishibashi and Yoshimasa Nishimura. 2024. Self-organized agents: A llm multi-agent framework toward ultra large-scale code generation and optimization. *arXiv preprint arXiv:2404.02183* (2024).
- [7] Sourena Khanzadeh. 2025. AgentMesh: A Cooperative Multi-Agent Generative AI Framework for Software Development Automation. (2025). *arXiv:2507.19902 [cs.SE]*
- [8] Haoran Li, Ziyi Su, Yun Xue, Zhiliang Tian, Yiping Song, and Minlie Huang. 2025. Advancing Collaborative Debates with Role Differentiation through Multi-Agent Reinforcement Learning. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 22655–22666. doi:10.18653/v1/2025.acl-long.1105
- [9] Leilei Lin, Yingming Zhou, Wenlong Chen, and Chen Qian. 2024. Think-on-process: Dynamic process generation for collaborative development of multi-agent system. *arXiv preprint arXiv:2409.06568* (2024).
- [10] Yuxuan Liu, Hongda Sun, Wei Liu, Jian Luan, Bo Du, and Rui Yan. 2025. MobileSteward: Integrating Multiple App-Oriented Agents with Self-Evolution to Automate Cross-App Instructions. *arXiv preprint arXiv:2502.16796* (2025).
- [11] Yohei Nakajima. 2023. BabyAGI. <https://github.com/yoheinakajima/babyagi>. GitHub repository.
- [12] Minh Huynh Nguyen, Thang Phan Chau, Phong X Nguyen, and Nghi DQ Bui. 2025. Agilecoder: Dynamic collaborative agents for software development based on agile methodology. In *2025 IEEE/ACM Second International Conference on AI Foundation Models and Software Engineering (Forge)*. IEEE, 156–167.
- [13] Timothy Ossowski, Jixuan Chen, Danyal Maqbool, Zefan Cai, Tyler Bradshaw, and Junjie Hu. 2024. Comma: A communicative multimodal multi-agent benchmark. *arXiv preprint arXiv:2410.07553* (2024).
- [14] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative Agents: Interactive Simulacra of Human Behavior. *arXiv:2304.03442 [cs.HC]* <https://arxiv.org/abs/2304.03442>
- [15] Chen Qian, Jiahao Li, Yufan Dang, Wei Liu, YiFei Wang, Zihao Xie, Weize Chen, Cheng Yang, Yingli Zhang, Zhiyuan Liu, et al. 2024. Iterative experience refinement of software-developing agents. *arXiv preprint arXiv:2405.04219* (2024).
- [16] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, et al. 2023. Chatdev: Communicative agents for software development. *arXiv preprint arXiv:2307.07924* (2023).
- [17] Qian Wang, Tianyu Wang, Zhenheng Tang, Qinbin Li, Nuo Chen, Jingsheng Liang, and Bingsheng He. 2025. MegaAgent: A large-scale autonomous LLM-based multi-agent system without predefined SOPs. In *Findings of the Association for Computational Linguistics: ACL 2025*. 4998–5036.
- [18] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. 2024. Autogen: Enabling next-gen LLM applications via multi-agent conversations. In *First Conference on Language Modeling*.
- [19] Hui Yang, Sifu Yue, and Yunzhong He. 2023. Auto-GPT for Online Decision Making: Benchmarks and Additional Opinions. *arXiv:2306.02224 [cs.AI]* <https://arxiv.org/abs/2306.02224>
- [20] Rui Ye, Shuo Tang, Rui Ge, Yaxin Du, Zhenfei Yin, Siheng Chen, and Jing Shao. 2025. MAS-GPT: Training LLMs to build LLM-based multi-agent systems. *arXiv preprint arXiv:2503.03686* (2025).
- [21] Sai Zhang, Zhenchang Xing, Ronghui Guo, Fangzhou Xu, Lei Chen, Zhaoyuan Zhang, Xiaowang Zhang, Zhiyong Feng, and Zhiqiang Zhuang. 2025. Empowering agile-based generative software development through human-ai teamwork. *ACM Transactions on Software Engineering and Methodology* (2025).